NO EXPERIENCE NEEDED

Docker, Hands-On

Nine small steps. You'll run your first container, look inside it, find out what Docker is really doing, build your own image, and ship it — all from the terminal.

2 hours

> **Nervous? Don't be.** Every command on this page is safe, and everything you create gets deleted at the end. If something breaks, that's fine — several steps are *designed* to break so you can see what the error looks like. Nothing here can damage your machine.

The plan

Nine steps, roughly two hours, with a few spare minutes built in because everyone gets stuck somewhere.

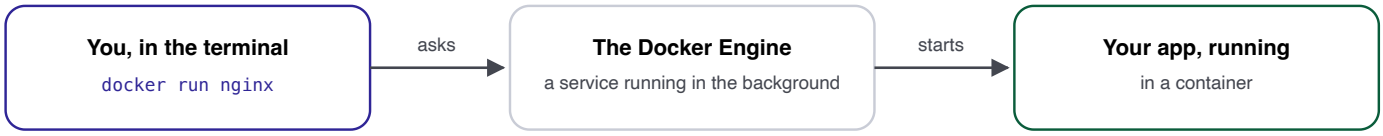
Min	Step	What you'll be able to say afterwards
0–8	Setup check + what a container is	"I know what I'm looking at"
8–14	**1** Run your first container	"I ran a web server without installing one"
14–22	**2** Images vs containers	"An image is the recipe, a container is the meal"
22–32	**3** Look inside a container	"It's just a normal process in disguise"
32–42	**4** Who actually does the work?	"The command I type does almost nothing"
42–50	**5** Setting limits	"I can cap a container's CPU and memory"
50–58	**6** Layers, and why containers are cheap	"100 containers cost barely more than one"
58–83	**7** Build your own image	"I can write a Dockerfile, and a good one"
83–93	**8** Keep your data	"My data survives the container"
93–111	**9** Ship it like a pro	"I built a production image and published it"
111–120	Recap + take-home	—

First: what *is* a container?

A container is **one program running on your computer, wrapped up so that it thinks it has the whole machine to itself.** That's it. That's the whole idea.

It is *not* a virtual machine. There is no second operating system booting up inside it. A container is an ordinary process — the same kind of thing as your browser or your text editor — that the Linux kernel has been told to keep in its own little room.

> 🏢 **The apartment analogy.** A **virtual machine** is building a whole new house next door: its own foundation, its own plumbing, its own everything. Slow to build, expensive, but completely separate.
>
> A **container** is renting a room in a building that already exists. You share the foundation and the plumbing (the **kernel**) with everyone else. You get a lock on your door so you can't see into other rooms, and a meter so you can't use all the electricity. Cheap, and ready in a second.
>
> Today you'll find the lock (**namespaces**, step 3) and the meter (**cgroups**, step 5) with your own hands.



The whole thing. Everything else today is just looking closely at that middle box.

Five words you'll hear all day

Word	In plain English
Image	A recipe. A frozen, read-only snapshot of an app and everything it needs. You download images; you don't run them directly.
Container	The meal you cooked from the recipe. A running (or stopped) instance of an image. One image → as many containers as you like.
Dockerfile	The written instructions for *making* a recipe. You'll write two in step 7.
Registry	An app store for images. Docker Hub is the public one; you'll run your own private one in step 9.
Volume	A USB stick you plug into a container. Anything written there survives after the container is deleted.

> ⚠️ **This lab needs a real Linux terminal.** Steps 3, 4 and 5 inspect the host's own processes and kernel settings, and **Docker Desktop hides all of that inside a virtual machine** — so Docker Desktop on its own will not do. On Windows you have two good ways to get real Linux (WSL2 or a VirtualBox VM) — see **Setup check** just below. On a lab PC running Ubuntu you're already set.

Setup check 2 MINUTES

On Windows? Open the 🪟 box below *first* — you need a real Linux terminal before any of this works. On a lab PC or an Ubuntu machine, carry straight on.

Docker should already be installed from the pre-lab. Let's make sure. Run these three:

```
docker version
systemctl is-active docker
docker run hello-world
```

YOU SHOULD SEE SOMETHING LIKE THIS

```
Client: Docker Engine - Community
 Version: 27.x
Server: Docker Engine - Community      <-- both Client AND Server must appear
active
Hello from Docker!
```

💬 Notice that `docker version` printed **two** sections: a **Client** and a **Server**. That's your first clue that "Docker" is actually two separate programs talking to each other. Step 4 is entirely about that.

Finally, pull down today's images now, so nobody is waiting on a download halfway through a step. If your instructor already did this, it finishes in seconds:

```
for i in hello-world alpine:3.20 nginx:alpine httpd:alpine python:3.12-slim registr
```

⚠️ **Got `permission denied ... docker.sock`?** Your user isn't in the `docker` group yet. Run `newgrp docker`, or log out and log back in. This is the single most common first-five-minutes problem.

▼ 🪟 On Windows? Open this first — get a real Linux terminal (do it before the lab)

Windows can't run this lab directly. You need Linux underneath. Two routes, both fine:

Route A — WSL2 (recommended: faster, lighter, about 20 minutes)

WSL2 runs a genuine Linux kernel inside Windows. Everything in this lab works on it.

1. Open **PowerShell as Administrator** (right-click Start → Terminal (Admin)) and run:

```
wsl --install -d Ubuntu-24.04
```

Reboot if it asks. When Ubuntu opens for the first time it asks you to pick a username and password — that password is what `sudo` will ask for later. Then confirm you are on version 2:

```
wsl -l -v
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
  NAME            STATE           VERSION
* Ubuntu-24.04    Running         2          <-- must say 2, not 1
```

2. Turn on systemd. Step 4 uses `systemctl` to stop and start the Docker service, and that needs systemd. *Inside the Ubuntu window*, run:

```
sudo tee /etc/wsl.conf > /dev/null <<'EOF'
[boot]
systemd=true
EOF
```

3. Make the kernel use cgroup v2. Without this, step 5 can't read its limits from `/sys/fs/cgroup/memory.max`, because WSL sets up the old and new systems side by side. In **PowerShell**, create the file `C:\Users\<your-name>\.wslconfig` containing:

```
[wsl2]
kernelCommandLine = cgroup_no_v1=all
```

4. Restart WSL so steps 2 and 3 take effect. In PowerShell:

```
wsl --shutdown
```

Reopen Ubuntu from the Start menu, then check systemd came up:

```
systemctl is-system-running
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
running          (or "degraded" - that is fine too)
```

5. Install Docker inside Ubuntu using the Linux instructions in the next box. Run them in your **Ubuntu window**, not in PowerShell.

> ⚠️ **Do not install Docker Desktop for this lab.** If it's already installed, open its *Settings* → *Resources* → *WSL integration* and switch this Ubuntu distro **off**. Otherwise your `docker` command talks to Docker Desktop's hidden daemon instead of your own, and steps 3, 4 and 5 will show you nothing.

Route B — Ubuntu in VirtualBox (slower, but exactly like the lab PCs)

1. Install **VirtualBox** from virtualbox.org and download the **Ubuntu 24.04 Desktop** ISO from ubuntu.com.
2. New VM → Linux / Ubuntu 64-bit → **4 GB RAM, 2 CPUs, 25 GB disk** → attach the ISO → install Ubuntu inside it.
3. Open a terminal in that Ubuntu and follow the Linux install instructions in the next box.

If the VM refuses to start or only offers 32-bit, turn on **virtualisation** (Intel VT-x / AMD-V) in your BIOS.

Then check you really are on real Linux

Whichever route you took, run this after installing Docker. It's the test that decides whether this lab will work:

```
ps -ef | grep dockerd | grep -v grep
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
root  312  1  0 09:14 ?  00:00:02 /usr/bin/dockerd -H fd:// ...
```

If you can see a real `dockerd` process, you're ready. **If it prints nothing** but `docker ps` still works, you are talking to Docker Desktop — go back and turn off its WSL integration.

> 💡 **Handy:** on WSL2, ports map straight through to Windows. When step 1 says open `http://localhost:8080`, you can open it in Chrome or Edge on Windows as usual.

▼ Docker not installed at all? Open this (about 15 minutes)

```
# 1. remove old conflicting packages (safe if they're not installed)
for pkg in docker.io docker-doc docker-compose podman-docker containerd runc;

# 2. add Docker's official software repository
sudo apt-get update
sudo apt-get install -y ca-certificates curl socat
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyri
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docke
sudo apt-get update

# 3. install Docker
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-pl

# 4. let your user run docker without sudo
sudo usermod -aG docker $USER
newgrp docker

# 5. download the images this lab uses, now, while you have time
for i in hello-world alpine:3.20 nginx:alpine httpd:alpine python:3.12-slim re
```

▼ Optional: record your terminal, so you can submit it as proof

```
mkdir -p ~/docker-lab && cd ~/docker-lab
script -a ~/docker-lab/lab-session.log
```

Everything you type from now on gets saved to that file. Type `exit` at the very end of the lab to stop recording.

Also: **open a second terminal window now**. A couple of steps need two.

---

1 Run your first container

Get a real web server running, without installing a web server.

~6 min Easy

Nginx is a professional web server. Normally you'd install it, configure it, start it. Instead:

```
docker run -d --name web -p 8080:80 nginx:alpine
```

Now open **http://localhost:8080** in your browser, or from the terminal:

```
curl -s http://localhost:8080 | grep -i "<title>"
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
<title>Welcome to nginx!</title>
```

> 💬 **You just ran a web server that isn't installed on your computer.** Nothing was added to your system's programs. Delete the container and every trace of nginx is gone. That's the promise of containers: software that arrives complete and leaves without a mess.

What did those flags mean?

That one command had four parts. Here's each one, and the others you'll meet today:

Flag	What it does
`-d`	**D**etached — run in the background and give you your prompt back. Without it, the container takes over your terminal.
`--name web`	Give it a name, so you can type `web` instead of a long random ID.
`-p 8080:80`	**P**ort. Traffic to port **8080 on your machine** goes to port **80 inside the container**. Left is yours, right is theirs.
`nginx:alpine`	The image to use. `alpine` after the colon is the **tag** — a version label. Here it means "the tiny version".
`--rm`	Delete the container the moment it stops. Handy for throwaway commands.
`-it`	Interactive terminal — use this when you want a shell inside the container.

The four commands you'll use constantly

```
docker ps                 # what's running right now
docker ps -a              # everything, including stopped containers
docker logs web           # what the app printed
docker stop web           # stop it (it still exists)
```

Stopped is not deleted. Check, then start it again:

```
docker ps -a --filter name=web --format "{{.Names}} -> {{.Status}}"
docker start web
curl -s -o /dev/null -w "back up: HTTP %{http_code}\n" http://localhost:8080
```

YOU SHOULD SEE SOMETHING LIKE THIS

```
web -> Exited (0) 12 seconds ago
back up: HTTP 200
```

⚠️ `docker stop` is **pause and keep**. `docker rm` is **delete forever**. Getting these two confused is how people lose work. Leave `web` running — the next step uses it.

---

2 Images vs containers

See that `docker run` is really three smaller commands in a trench coat.

~8 min · Easy

People mix up images and containers constantly. The fastest cure is to do each stage separately and watch what appears.

Stage 1 — pull: the recipe arrives

```
docker pull httpd:alpine
docker images httpd
docker ps -a --filter ancestor=httpd:alpine     # empty!
```

You now *have* the image. But no container exists. A recipe in your kitchen isn't dinner.

Stage 2 — create: prepared, but not cooking

```
docker create --name demo -p 8081:80 httpd:alpine
docker ps -a --filter name=demo --format "table {{.Names}}\t{{.Status}}"
curl -s http://localhost:8081 || echo ">>> nothing is listening - the container ex
```

YOU SHOULD SEE SOMETHING LIKE THIS

```
NAMES   STATUS
demo    Created
>>> nothing is listening - the container exists but isn't running
```

Stage 3 — start: now it's alive

```
docker start demo
curl -s http://localhost:8081 | head -n 1
```

YOU SHOULD SEE SOMETHING LIKE THIS

```
<html><body><h1>It works!</h1></body></html>
```

💬 `docker run` = `pull` + `create` + `start`. That's all it ever was. It's a shortcut that does the three things you just did by hand. Remember this — in step 4 you'll find a situation where the shortcut isn't available and you'll need the pieces.

One image, many containers

Ask for the same name twice and Docker refuses, because `run` always tries to *create* something new:

```
docker run -d --name demo httpd:alpine
```

YOU SHOULD SEE SOMETHING LIKE THIS

```
docker: Error response from daemon: Conflict. The container name "/demo" is
already in use by container "a1b2c3...". You have to remove (or rename) that
container to be able to reuse that name.
```

Drop the name, though, and you can have as many as you like from the one image:

```
docker run -d httpd:alpine
docker run -d httpd:alpine
docker ps --filter ancestor=httpd:alpine --format "{{.ID}}  {{.Names}}"
```

Three containers, one recipe. Step 6 explains why that costs almost no disk space.

Stopping politely vs stopping rudely

```
docker run -d --name sleeper1 alpine:3.20 sleep 1000
docker run -d --name sleeper2 alpine:3.20 sleep 1000

time docker stop sleeper1      # polite: "please finish up"
time docker kill sleeper2      # rude: pulls the plug
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
real    0m10.2s     <- stop waited ten whole seconds
real    0m0.1s      <- kill was instant
```

> 💬 **Why did the polite one take ten seconds?** `docker stop` sends a signal that means "please shut down cleanly" and then waits up to 10 seconds before forcing it. The `sleep` program ignores that signal entirely, so Docker waited the full 10 seconds and then killed it anyway.
>
> This looks like a curiosity. It isn't — it's one of the most common real bugs in production, and **you will cause it yourself, on purpose, in step 7**, then fix it.

Tidy up before moving on (keep `web`):

```
docker rm -f sleeper1 sleeper2
docker rm -f $(docker ps -aq --filter ancestor=httpd:alpine)
```

---

3 Look inside a container

Open the door, walk in, and discover it's just a normal process wearing a disguise.

~10 min | Easy | the "oh!" moment

Your `web` container from step 1 should still be running. Let's get a shell inside it:

```
docker exec -it web sh
```

Your prompt changes. **You are now inside the container.** Try these *in there*:

```
hostname          # a strange short name - that's the container ID
ps                # barely any processes, and nginx is number 1
ls /              # a whole Linux filesystem... but not YOUR filesystem
cat /etc/os-release | head -n 1
exit              # back to your own machine
```

> 💬 **Look at what `ps` showed.** On your real machine there are hundreds of processes. In there, about two — and nginx is **PID 1**, the very first process. The container genuinely believes it just booted up and that nginx is the only thing alive.
>
> It's lying to itself, and the kernel is helping.

The same process, seen from outside

Back on your own machine, ask Docker for that process's *real* ID:

```
PID=$(docker inspect -f '{{.State.Pid}}' web)
echo "inside the container it thinks it is PID 1. Out here it is PID $PID"
ps -o pid,user,cmd -p $PID
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
inside the container it thinks it is PID 1. Out here it is PID 24811
  PID USER  CMD
24811 root  nginx: master process nginx -g daemon off;
```

> 💬 **There it is, in your own process list.** Not hidden in a virtual machine — sitting in `ps` next to your text editor. A container is a normal Linux process that has been told a few lies about the world.
>
> The lies are called **namespaces**. There's one for process IDs (so it sees itself as PID 1), one for the hostname, one for the network, one for the filesystem. Same process, different view.

The proof that it isn't a virtual machine

If a container were a VM, it would be running its own operating system kernel. Check:

```
echo "your system  : $(. /etc/os-release && echo $PRETTY_NAME)"
echo "your kernel  : $(uname -r)"
docker exec web cat /etc/os-release | head -n 1
echo "its kernel   : $(docker exec web uname -r)"
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
your system  : Ubuntu 24.04.1 LTS
your kernel  : 6.8.0-45-generic
PRETTY_NAME="Alpine Linux v3.20"
its kernel   : 6.8.0-45-generic        <-- EXACTLY the same
```

💬 A **different Linux distribution** (Alpine, not Ubuntu) but **the exact same kernel** — because there's only one kernel, yours, shared by everything. That single fact is why a container starts in a fraction of a second while a VM takes a minute: there's no second operating system to boot.

▼ ▸ **Go deeper: see the namespaces for yourself**

A namespace is an ID number the kernel hands out. Compare the container's with your shell's — different numbers mean different worlds:

```
sudo readlink /proc/$PID/ns/pid /proc/$PID/ns/net /proc/$PID/ns/uts
readlink /proc/$$/ns/pid /proc/$$/ns/net /proc/$$/ns/uts
sudo lsns -p $PID
```

And the party trick — `nsenter` runs *your* programs inside the container's namespaces. The container has no `ip` command of its own, yet this works:

```
sudo nsenter -t $PID -u hostname
sudo nsenter -t $PID -n ip addr
```

Which tells you something important: the container isn't a box with the tools in it. **The namespace *is* the container.**

📌 Keep `web` running a little longer — step 4 needs it.

4 Who actually does the work?

Find out that the `docker` command you've been typing does almost nothing at all.

~10 min · A bit deeper

Remember `docker version` printing a **Client** and a **Server**? Those are two different programs. The `docker` command you type is only the **client**. It packages your request and hands it over. The **server** — a background service called `dockerd`, the "daemon" — does all the real work.

They talk to each other through a special file called a **socket**:

```
ls -l /var/run/docker.sock
ps -ef | grep -E "dockerd|containerd" | grep -v grep
```

Proving it: run a container with no Docker command at all

If the daemon does the work, we should be able to skip the `docker` command entirely and talk to the daemon ourselves. `curl` is a general-purpose tool for sending web requests — and the daemon speaks plain HTTP.

```
# this is exactly what "docker version" does under the hood
curl -s --unix-socket /var/run/docker.sock http://localhost/version | python3 -m js
```

Now the real thing. Create a container, then start it — **two web requests, no `docker run`**:

```
docker pull alpine:3.20

# request 1 - this is "docker create"
curl -s --unix-socket /var/run/docker.sock \
  -H "Content-Type: application/json" \
  -d '{"Image": "alpine:3.20", "Cmd": ["sleep", "300"]}' \
  -X POST "http://localhost/containers/create?name=api-made"
echo

# request 2 - this is "docker start"
curl -s -o /dev/null -w "HTTP status: %{http_code}\n" \
  --unix-socket /var/run/docker.sock \
  -X POST http://localhost/containers/api-made/start

docker ps --filter name=api-made
```

YOU SHOULD SEE SOMETHING LIKE THIS

```
{"Id":"3c8f0a1e5d...","Warnings":[]}
HTTP status: 204
CONTAINER ID   IMAGE          COMMAND       STATUS         NAMES
3c8f0a1e5d2b   alpine:3.20    "sleep 300"   Up 2 seconds   api-made
```

💬 **You started a container with `curl`.** The `docker` command was never involved. This is why it's useful to know that `run` = `pull` + `create` + `start`: here you had to do the pull yourself, because the daemon's create request won't download images for you — only the friendly `docker run` shortcut does that.

It also explains how tools like Kubernetes, VS Code and your CI server control Docker: they just send the same requests.

Take the daemon away and watch the CLI become useless

```
sudo systemctl stop docker.socket docker.service
docker ps
systemctl is-active containerd
```

YOU SHOULD SEE SOMETHING LIKE THIS

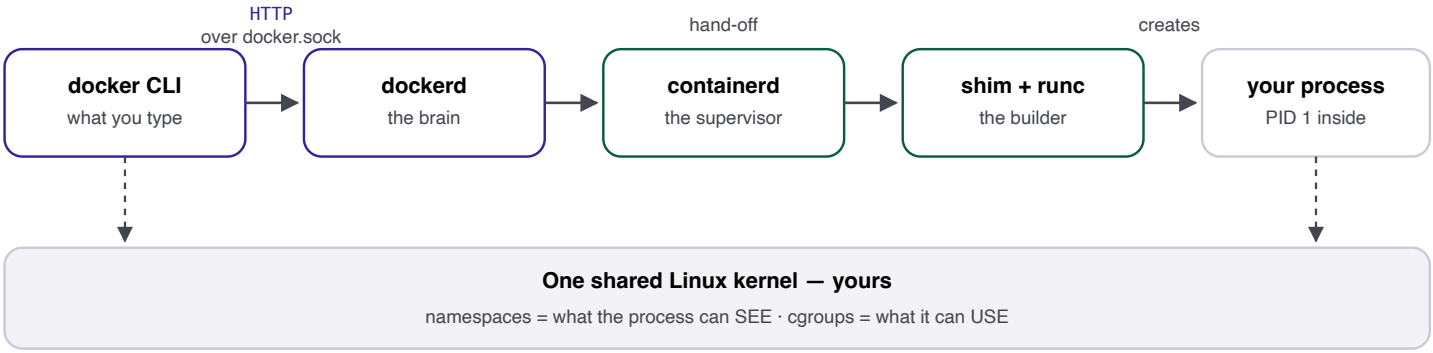
```
Cannot connect to the Docker daemon at unix:///var/run/docker.sock.
Is the docker daemon running?
active
```

The `docker` command is still installed — it just has nobody to talk to. But look at that last line: **`containerd` is still running.** It's yet another separate program. Put Docker back:

```
sudo systemctl start docker
docker ps
docker rm -f api-made
```

So how many programs are involved?

Five, it turns out. Each hands off to the next:



You've now met boxes 1 and 2. Let's find 3 and 4 in the process list.

Finding the last two in ps

```
PID=$(docker inspect -f '{{.State.Pid}}' web)

# who is the container's PARENT process?
ps -o pid,cmd -p $(ps -o ppid= -p $PID)

# and where is runc, the thing that supposedly built it?
pgrep -a runc || echo ">>> runc is GONE - it built the container and exited"
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
  PID CMD
24790 /usr/bin/containerd-shim-runc-v2 -namespace moby -id 6f1c...
>>> runc is GONE - it built the container and exited
```

💬 **Two surprises in one screen.**

runc is a builder, not a manager. It sets up the namespaces and limits, starts your process, and immediately quits. It is never running while your container is.

The **shim** is what stays behind as the parent. That's why you can restart Docker itself without killing all your running containers — they aren't children of `dockerd` at all.

▼ ▸ **Go deeper: ask containerd directly, and watch the raw HTTP**

Docker files its containers under a containerd namespace called `moby`:

```
sudo ctr namespaces list
sudo ctr -n moby tasks list
```

Compare the ID and PID with `docker ps --no-trunc`. Same container, one level down. (If `ctr` can't find its socket, add `--address /var/run/docker/containerd/containerd.sock`.)

Watch the CLI's actual HTTP traffic. In terminal 2, start a proxy that prints everything passing through:

```
sudo socat -v UNIX-LISTEN:/tmp/docker-proxy.sock,fork,mode=777 UNIX-CONNECT:/v
```

In terminal 1, point the CLI at the proxy:

```
docker -H unix:///tmp/docker-proxy.sock ps
```

Terminal 2 fills with ordinary web requests: `GET /v1.47/containers/json HTTP/1.1`. Press `Ctrl+C` when you've seen enough.

> 🔐 **Worth knowing:** the daemon runs as root, and anyone who can write to `/var/run/docker.sock` can tell it what to do. Being in the `docker` group is effectively being root on that machine. Remember that before you add someone to it.

---

5 Setting limits

Stop one container from eating the whole machine.

~8 min · 🟡 A bit deeper

Namespaces control what a container can *see*. A second kernel feature, **cgroups** (control groups), controls what it can *use* — the electricity meter from the apartment analogy.

Ask for a limit, then check the kernel really got it

```
docker run -d --name limited --cpus=0.5 --memory=128m nginx:alpine

# what you asked Docker for
docker inspect -f 'Memory={{.HostConfig.Memory}} NanoCpus={{.HostConfig.NanoCpus}}'

# what the kernel is actually enforcing, read from inside the container
docker exec limited cat /sys/fs/cgroup/memory.max
docker exec limited cat /sys/fs/cgroup/cpu.max
```

YOU SHOULD SEE SOMETHING LIKE THIS

```
Memory=134217728 NanoCpus=500000000
134217728          <- 128 MB, in bytes
50000 100000       <- "50ms of CPU every 100ms" = half a CPU
```

💬 Your friendly `--cpus=0.5` turned into two numbers in a file that the Linux kernel reads. Docker didn't build a cage — it filled in a form, and the kernel does the enforcing.

A runaway container, caught

This container does nothing but burn CPU in a loop forever — the classic "one bad tenant slows down the whole server" problem:

```
docker run -d --name burner --cpus=0.5 alpine:3.20 sh -c 'while :; do :; done'
sleep 5
docker stats --no-stream burner
```

It sits at about **50%**, exactly as told. And you can change your mind without restarting it:

```
docker update --cpus=1 burner
sleep 5
docker stats --no-stream burner          # now about 100%
```

Hitting the memory ceiling

Here's a container allowed 64 MB, trying to grab 200 MB:

```
docker run --name oom-test --memory=64m --memory-swap=64m python:3.12-slim \
  python -c "data = b'x' * (200 * 1024 * 1024); print('allocated')"
echo "exit code: $?"
docker inspect -f 'OOMKilled={{.State.OOMKilled}} ExitCode={{.State.ExitCode}}' oom
```

YOU SHOULD SEE SOMETHING LIKE THIS

```
exit code: 137
OOMKilled=true ExitCode=137
```

> 💬 The word `allocated` **never printed** — the program was killed mid-sentence. "OOM" is **Out Of Memory**, and `OOMKilled=true` is the kernel admitting it pulled the trigger.
>
> **Exit code 137 is worth memorising.** It means "killed by force". You saw it in step 2 as well, when `docker stop` gave up waiting. Any time a container mysteriously dies with 137, your first question is: was it memory, or was it a shutdown that took too long?

```
docker rm -f limited burner oom-test
```

▼ ▸ Go deeper: fork-bomb protection, and the kernel's own log

```
# this container may only ever have 10 processes
docker run --rm --pids-limit=10 alpine:3.20 sh -c 'for i in $(seq 1 20); do sl

# the kernel's record of the OOM kill you just caused
sudo dmesg | grep -i -E "oom|killed process" | tail -n 3
```

You'll see `sh: can't fork: Resource temporarily unavailable` — a fork bomb that would normally freeze a machine, shrugged off.

---

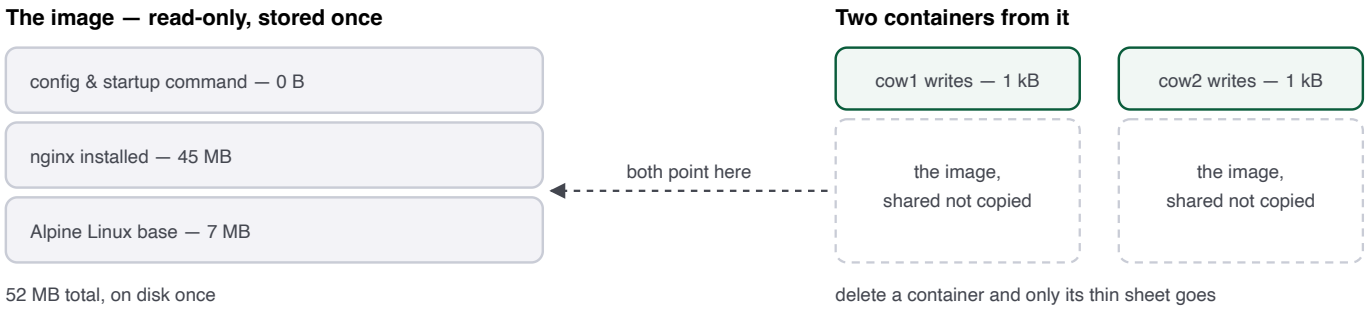
6 Layers, and why containers are so cheap

Change a file in one container and watch its twin stay untouched.

~8 min · 🟡 A bit deeper

An image isn't one big blob. It's a **stack of read-only layers**, like sheets of tracing paper laid on top of each other — you look down through the stack and see one combined picture.

When you start a container, Docker doesn't copy that stack. It puts **one thin writable sheet on top**, and that sheet is the only thing that belongs to your container.



100 containers = 52 MB + 100 kB. Not 5.2 GB.

This is called copy-on-write: nothing is copied until something is written.

See it happen

```
docker run -d --name cow1 nginx:alpine
docker run -d --name cow2 nginx:alpine

# overwrite nginx's home page - but only inside cow1
docker exec cow1 sh -c 'echo "<h1>Changed in cow1</h1>" > /usr/share/nginx/html/inc

docker exec cow1 cat /usr/share/nginx/html/index.html
docker exec cow2 cat /usr/share/nginx/html/index.html | grep -i "<title>"
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
<h1>Changed in cow1</h1>
<title>Welcome to nginx!</title>     <-- cow2 never noticed
```

That file lives in a *read-only* layer that both containers share. cow1 didn't edit it — it made its own copy on its own sheet, and now sees that instead.

What it cost

```
docker diff cow1
docker ps -s --filter name=cow --format "table {{.Names}}\t{{.Size}}"
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
C /usr/share/nginx/html/index.html    <- C = changed, A = added, D = deleted
A /run/nginx.pid
NAMES   SIZE
cow1    1.2kB (virtual 52MB)
cow2    1.1kB (virtual 52MB)
```

> 💬 Two numbers per container. The first (~1 kB) is **what this container owns**. The "virtual" 52 MB is **what it can see**, almost all of it shared with its twin. That gap is the reason you can run dozens of containers on a laptop.

The catch: that thin sheet is disposable

```
# stopping and starting keeps your change...
docker stop cow1 && docker start cow1
docker exec cow1 cat /usr/share/nginx/html/index.html

# ...but deleting the container throws the sheet away
docker rm -f cow1
docker run -d --name cow1 nginx:alpine
docker exec cow1 cat /usr/share/nginx/html/index.html | grep -i "<title>"
```

Back to the original. Your change is gone for good.

> ⚠️ **This is the mistake that costs people real data.** Anything your app writes — uploads, a database, logs — lives on that disposable sheet and dies with the container. Step 8 is the fix, and it's the reason step 8 exists.

```
docker rm -f cow1 cow2 web
```

---

7 Build your own image

Write a Dockerfile. Then write a better one, and measure exactly how much better.

~25 min | 🟡 The main event | take your time here

Until now you've used other people's images. Now you'll package an app of your own — a small **visit counter** web service. **You don't need to know Python.** The code is written for you; your job is the packaging.

7.1 Set up the folder

```
mkdir -p ~/docker-lab/campusconnect && cd ~/docker-lab/campusconnect
echo "flask==3.0.3" > requirements.txt
```

Now the app itself. Copy this **entire block** — it writes the file for you:

▼ Click to open app.py — copy the whole thing into your terminal

```
cat > app.py <<'EOF'
import argparse, os, signal, socket, sys
from flask import Flask, jsonify

app = Flask(__name__)
DATA_DIR = os.environ.get("DATA_DIR", "/data")
COUNT_FILE = os.path.join(DATA_DIR, "visits.txt")
APP_VERSION = os.environ.get("APP_VERSION", "v1")

def read_count():
    try:
        with open(COUNT_FILE) as f:
            return int(f.read().strip() or 0)
    except FileNotFoundError:
        return 0

def write_count(n):
    os.makedirs(DATA_DIR, exist_ok=True)
    with open(COUNT_FILE, "w") as f:
        f.write(str(n))

@app.route("/")
def index():
    n = read_count() + 1
    write_count(n)
    return jsonify(message="Hello from CampusConnect!", visits=n,
                   hostname=socket.gethostname(), pid=os.getpid(),
                   version=APP_VERSION)

@app.route("/health")
def health():
    return jsonify(status="ok")

def handle_sigterm(signum, frame):
    print("Received SIGTERM — shutting down gracefully", flush=True)
    sys.exit(0)

signal.signal(signal.SIGTERM, handle_sigterm)

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
```

```
    parser.add_argument("--port", type=int, default=8000)
    args = parser.parse_args()
    print(f"Starting CampusConnect {APP_VERSION} on port {args.port}", flush=T
    app.run(host="0.0.0.0", port=args.port)
EOF
```

All you need to know about it:

- Visit `/` and it adds 1 to a counter saved in `/data/visits.txt`, then tells you the count, the container's hostname and its PID.
- Visit `/health` and it says `{"status":"ok"}`.
- When asked to shut down politely, it prints *"Received SIGTERM - shutting down gracefully"* and exits. **Watch for that line later — it's the whole point of 7.4.**

7.2 What goes in a Dockerfile

A Dockerfile is a plain text list of build steps. There are only about eight instructions you'll ever use much:

Instruction	What it means
`FROM`	Start from this existing image. Always the first line.
`WORKDIR`	Work inside this folder from here on (like `cd`).
`COPY`	Copy files from your folder into the image.
`RUN`	Run a command **while building** — installing packages, mostly.
`ENV`	Set an environment variable inside the image.
`EXPOSE`	A note to humans: "this app listens on this port". It doesn't actually open anything.
`ENTRYPOINT`	The program that runs when the container starts.
`CMD`	Default arguments for that program — replaceable when you run it.

Write your first one:

```
cat > Dockerfile.v1 <<'EOF'
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 8000
CMD echo "Booting CampusConnect..."; python app.py
EOF

time docker build -t campusconnect:v1 -f Dockerfile.v1 .
docker run -d --name cc-v1 -p 8080:8000 campusconnect:v1
sleep 2
curl -s http://localhost:8080/; echo
```

YOU SHOULD SEE SOMETHING LIKE THIS

```
{"hostname":"992b057cfe34","message":"Hello from CampusConnect!","pid":7,"version"
```

It works. Genuinely — you packaged an app. But it has two bugs that you can't see yet, and both are extremely common. Let's find them.

7.3 Bug one: every rebuild is slow

```
# rebuild with nothing changed — everything is cached, about a second
time docker build -t campusconnect:v1 -f Dockerfile.v1 .

# now change a single word of the app
sed -i 's/Hello from CampusConnect!/Hello from CampusConnect (edited)!/' app.py

# rebuild, and watch "pip install" run all over again
time docker build -t campusconnect:v1 -f Dockerfile.v1 .
```

💬 **Why?** Docker caches each instruction as a layer. But the moment one layer changes, **everything below it must be rebuilt too** — the cache can't be trusted any more.

Our `COPY . .` copies the app code *before* `pip install`. So editing one character of code invalidates the copy step, which forces the install step to run again. With 40 dependencies that's minutes, on every single save.

7.4 Bug two: it can't shut down properly

```
docker top cc-v1 -o pid,ppid,cmd
time docker stop cc-v1
docker inspect -f 'exit code: {{.State.ExitCode}}' cc-v1
docker logs cc-v1
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
PID    PPID   CMD
3182   3157   /bin/sh -c echo "Booting CampusConnect..."; python app.py
3209   3182   python app.py
real    0m10.2s
exit code: 137
Booting CampusConnect...      <-- no "Received SIGTERM" line. It never ran.
```

💬 **Ten seconds, and exit code 137 again.** Same symptom as the `sleep` container in step 2, and now you can see why.

Look at that `ps` output: `/bin/sh` **is PID 1, not your app.** Writing `CMD echo ...; python app.py` without square brackets is called **shell form**, and it wraps your command in a shell. Docker politely asks PID 1 to shut down — but PID 1 is the shell, and the shell doesn't pass the message on. Python never hears it, waits out the 10 seconds, and gets force-killed mid-work.

⚠️ Now picture that container being a database, force-killed in the middle of writing, on every single deployment.

7.5 Fix both

Two changes: **dependencies before code**, and **square brackets**.

```
cat > Dockerfile.v2 <<'EOF'
FROM python:3.12-slim
WORKDIR /app

# FIX 1: dependencies first. They rarely change, so this layer stays cached.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# app code last, because it changes constantly
COPY app.py .

ENV APP_VERSION=v2
EXPOSE 8000

# FIX 2: square brackets = "exec form". Python becomes PID 1 itself.
ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8000"]
EOF

time docker build -t campusconnect:v2 -f Dockerfile.v2 .

# edit the code again - pip should now say CACHED
sed -i 's/(edited)/(edited twice)/' app.py
time docker build -t campusconnect:v2 -f Dockerfile.v2 .
```

Now run it and try shutting it down:

```
docker run -d --name cc-v2 -p 8082:8000 campusconnect:v2
sleep 2
curl -s http://localhost:8082/; echo
docker top cc-v2 -o pid,ppid,cmd
time docker stop cc-v2
docker inspect -f 'exit code: {{.State.ExitCode}}' cc-v2
docker logs cc-v2
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
{"hostname":"895d354595dc","pid":1,...}      <-- pid is 1 now, not 7
PID    PPID   CMD
4547   4521   python app.py --port 8000     <-- no /bin/sh anywhere
real    0m0.2s                               <-- instead of 10 seconds
exit code: 0                                 <-- clean, instead of 137
Starting CampusConnect v2 on port 8000
Received SIGTERM - shutting down gracefully <-- it finally got the message
```

The two rules to take away

1. **Put the things that rarely change at the top of your Dockerfile**, and the things that change constantly at the bottom. Dependencies before code, always.
2. **Always use square brackets:** `ENTRYPOINT ["python", "app.py"]`, never `CMD python app.py`. Brackets mean your app is PID 1, and PID 1 is who Docker talks to.

7.6 ENTRYPOINT vs CMD, in one demo

`ENTRYPOINT` is the program. `CMD` is its default arguments — and **anything you type after the image name replaces them**:

```
docker run -d --name cc-alt -p 9000:9000 campusconnect:v2 --port 9000
sleep 2
curl -s http://localhost:9000/; echo
docker logs cc-alt | head -n 1
```

Same image, different port, no rebuild. That's how one image serves every environment.

```
docker rm -f cc-v1 cc-v2 cc-alt
```

▼ ▸ Go deeper: a 30-second ENTRYPOINT + CMD playground

```
mkdir -p ~/docker-lab/greeter && cd ~/docker-lab/greeter
cat > Dockerfile <<'EOF'
FROM alpine:3.20
ENTRYPOINT ["ping"]
CMD ["-c", "3", "127.0.0.1"]
EOF
docker build -t pingtool:v1 .

docker run --rm pingtool:v1                       # ping -c 3 127.0.0.1
docker run --rm pingtool:v1 -c 1 localhost        # your args replace CMD
docker run --rm --entrypoint echo pingtool:v1 hi  # --entrypoint replaces the
cd ~/docker-lab/campusconnect
```

---

8 Keep your data

Lose the counter, then make it survive anything.

~10 min | Easy | fixes the step 6 problem

8.1 Watch the data disappear

```
cd ~/docker-lab/campusconnect
docker run -d --name cc -p 8080:8000 campusconnect:v2
sleep 2
for i in 1 2 3; do curl -s http://localhost:8080/; echo; done
```

Visits 1, 2, 3. Now delete the container and start a fresh one:

```
docker rm -f cc
docker run -d --name cc -p 8080:8000 campusconnect:v2
sleep 2
curl -s http://localhost:8080/; echo
docker rm -f cc
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
{"visits":1, ...}      <-- back to 1. Three visits, gone.
```

Exactly what step 6 predicted: the counter lived on the container's disposable sheet.

8.2 Plug in a volume

A **volume** is storage that Docker manages separately from any container — the USB stick. Create one and mount it at `/data`, which is where the app saves its counter:

```
docker volume create cc-data

docker run -d --name cc -p 8080:8000 -v cc-data:/data campusconnect:v2
sleep 2
for i in 1 2 3; do curl -s http://localhost:8080/; echo; done

# now DESTROY the container completely and build a brand new one
docker rm -f cc
docker run -d --name cc -p 8080:8000 -v cc-data:/data campusconnect:v2
sleep 2
curl -s http://localhost:8080/; echo
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
{"hostname":"b41d7e0a9c12", "visits":4, ...}
         ^ a different container       ^ but the data carried on
```

💬 **New container, same data.** The `-v cc-data:/data` part reads as: "whenever anything writes to `/data` inside, put it in the volume named `cc-data` instead."

This split — **throwaway containers, permanent volumes** — is how real systems are run. You replace containers freely on every deploy, and the data never moves.

8.3 Where does it actually live?

```
docker volume inspect -f '{{.Name}} -> {{.Mountpoint}}' cc-data
sudo cat /var/lib/docker/volumes/cc-data/_data/visits.txt; echo

# any other container can plug into the same volume
docker run --rm -v cc-data:/data alpine:3.20 cat /data/visits.txt; echo
```

8.4 Bind mounts: use your own folder instead

If you want to see the files on your own machine — very handy while developing — mount a folder instead of a volume:

```
mkdir -p ~/docker-lab/campusconnect/hostdata
docker run -d --name cc-bind -p 8083:8000 -v ~/docker-lab/campusconnect/hostdata:/d
sleep 2
curl -s http://localhost:8083/ > /dev/null
cat ~/docker-lab/campusconnect/hostdata/visits.txt; echo
```

The file is right there in your folder, readable in any editor.

> ⚠️ Bind mounts need a **full path**. Writing `-v hostdata:/data` doesn't use your folder at all — it quietly creates a *volume* called "hostdata" and your files go missing. Use `$(pwd)/hostdata` or `~/...`.

```
docker rm -f cc cc-bind      # keep the cc-data volume - step 9 uses it
```

---

9 Ship it like a pro

Turn your image into one you'd actually deploy, then publish it.

~18 min · The finish line

Your v2 image works, but it has three problems you'd be told off for in a real job:

- It runs as **root**. If someone breaks into the app, they're root inside the container.
- It carries **build leftovers** — pip's machinery, cached files — that the running app never needs.
- Docker has **no idea whether the app is actually working**. A frozen app still counts as "Up".

Three fixes, one Dockerfile. Read the comments as you paste it:

```
cd ~/docker-lab/campusconnect

cat > Dockerfile <<'EOF'
# ---------- stage 1: a workshop, thrown away at the end ----------
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---------- stage 2: the real image, starts clean ----------
FROM python:3.12-slim

# create a normal user with no special powers, and a folder it owns
RUN useradd --create-home --uid 10001 appuser \
    && mkdir -p /data \
    && chown appuser:appuser /data

# take ONLY the finished packages from the workshop. No pip leftovers.
COPY --from=builder /install /usr/local

WORKDIR /app
COPY app.py .

ENV APP_VERSION=v3 \
    PYTHONUNBUFFERED=1

# from here on, everything runs as appuser - not root
USER appuser
EXPOSE 8000

# let Docker check every 10s whether the app really answers
HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
  CMD ["python", "-c", "import urllib.request; urllib.request.urlopen('http://loca

ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8000"]
EOF

docker build -t campusconnect:v3 .
```

> 💬 **That "two FROM lines" trick is called a multi-stage build.** The first stage is a workshop: it installs everything, makes a mess, and is then thrown away. The second stage is the image you ship, and it only receives the finished parts. Smaller image, nothing extra to attack.

9.1 A real bug, on purpose

```
docker run -d --name cc -p 8080:8000 -v cc-data:/data --cpus=0.5 --memory=128m camp
sleep 3
curl -s -o /dev/null -w "HTTP %{http_code}\n" http://localhost:8080/
docker logs cc 2>&1 | grep -i -m1 permission
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
HTTP 500
PermissionError: [Errno 13] Permission denied: '/data/visits.txt'
```

> 💬 **Don't panic — this is supposed to happen, and it's a lesson.** The `visits.txt` in your volume was created back in step 8 by v2, which ran as **root**. Your new image runs as **appuser**, who isn't allowed to touch root's files.
>
> This trips up nearly everyone the first time they stop running containers as root. The fix is to change who owns the existing data — using a throwaway container as a tool:

```
docker run --rm -v cc-data:/data alpine:3.20 chown -R 10001:10001 /data
docker restart cc
sleep 3
curl -s http://localhost:8080/; echo
```

Working — and the counter carries on from where step 8 left it.

9.2 Check your three fixes actually landed

```
docker exec cc id                                  # who am I? not root
sleep 12
docker inspect -f 'health: {{.State.Health.Status}}' cc
docker images campusconnect                        # compare v2 and v3 sizes
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
health: healthy
```

`docker ps` now shows `(healthy)` next to this container. Docker is checking the app every 10 seconds, not just assuming.

9.3 Publish it to a registry

A **registry** is where images live so other machines can download them. Docker Hub is the public one. You're going to run your own, in a container, because of course it's a container:

```
docker run -d --name registry -p 5000:5000 registry:2

# "tag" = give the image a name that says where it belongs
docker tag campusconnect:v3 localhost:5000/campusconnect:v3
docker push localhost:5000/campusconnect:v3

# ask the registry what it's holding
curl -s http://localhost:5000/v2/_catalog; echo
curl -s http://localhost:5000/v2/campusconnect/tags/list; echo
```

```
YOU SHOULD SEE SOMETHING LIKE THIS
{"repositories":["campusconnect"]}
{"name":"campusconnect","tags":["v3"]}
```

9.4 Deploy it the way a real server would

Delete every local copy, then pull it back as if you were a brand-new machine:

```
docker rm -f cc
docker rmi localhost:5000/campusconnect:v3 campusconnect:v3

docker pull localhost:5000/campusconnect:v3
docker run -d --name cc -p 8080:8000 -v cc-data:/data --cpus=0.5 --memory=128m loca
sleep 3
curl -s http://localhost:8080/; echo
```

> 💬 **Look closely at the `pull` output: most lines say `Already exists`.** The Python base layers were still on your disk from earlier, so only your few kilobytes of app came across the network. That's layers paying off again.
>
> And this is the whole professional workflow you just performed by hand: **build once → push to a registry → pull and run anywhere**. The bytes running in production are byte-for-byte the ones you tested. No more "but it works on my machine".

🎉 **That's the lab.** Clean up whenever you're ready:

```
docker rm -f $(docker ps -aq) 2>/dev/null
docker volume rm cc-data 2>/dev/null
docker image prune -f
```

> ⚠️ On a shared lab machine, **don't** run `docker system prune -a`. It deletes the pre-downloaded images and the next batch of students will spend the session waiting for downloads.

✅ What you learned

Nine steps, nine sentences. If you can say these, you've got it.

1. A container is **one process** on your machine, wrapped so it thinks it's alone. You ran a web server without installing one.
2. `docker run` is just `pull` + `create` + `start`. **An image is the recipe; a container is the meal.**
3. Inside, your app is PID 1; outside, it's a normal process in `ps`. Same kernel, inside and out — **a container is not a virtual machine**.
4. The `docker` command is only a messenger. A background service does the work — you proved it by starting a container with `curl`.
5. **Namespaces** decide what a container can see; **cgroups** decide what it can use. Exit code **137** means something was force-killed.
6. Images are **shared read-only layers**; each container adds one thin writable sheet, and that sheet is deleted with the container.
7. In a Dockerfile: **rarely-changing things first**, and **always use square brackets** so your app is PID 1 and can shut down cleanly.
8. **Volumes** hold data that must outlive containers.
9. A production image is **multi-stage, non-root and health-checked**, and it lives in a **registry**: build once, run anywhere.

Commands worth keeping

When you want to...	Run
See what's running / everything	`docker ps` / `docker ps -a`
Read an app's output	`docker logs NAME`
Get a shell inside	`docker exec -it NAME sh`
See what a container changed	`docker diff NAME`
Compare own size vs shared size	`docker ps -s`
Change limits without restarting	`docker update --cpus=1 NAME`
Find out why it died	`docker inspect -f '{{.State.ExitCode}} {{.State.OOMKilled}}' NAME`
Delete a running container	`docker rm -f NAME`
Start completely fresh	`docker rm -f $(docker ps -aq)`

If something goes wrong

What you see	What's happening	What to do
`permission denied ... docker.sock`	You're not in the `docker` group	`newgrp docker`, or log out and back in
`port is already allocated`	A container from an earlier step is still using it	`docker ps -a`, then `docker rm -f NAME`
`name is already in use`	That name is taken, even by a stopped container	`docker rm -f NAME`, or pick another name
`bash: not found`	Alpine images don't include bash	Use `sh` instead
Exit code **137**	Force-killed: ran out of memory, or ignored the shutdown request	Check `OOMKilled`; if false, it's the PID 1 problem from step 7
`$PID` is empty	New terminal, or the container stopped	Re-run the `PID=$(docker inspect ...)` line
`Cannot connect to the Docker daemon`	The daemon is stopped	`sudo systemctl start docker`
`toomanyrequests` when pulling	Docker Hub limits downloads per network, and the whole lab shares one	Tell your instructor; use the pre-downloaded images
`cat: /sys/fs/cgroup/memory.max: No such file`	The host is using the older cgroup v1 (common on WSL2 without the `.wslconfig` tweak)	Check with `docker info --format '{{.CgroupVersion}}'`. On v1 read `/sys/fs/cgroup/memory/memory.limit_in_bytes` instead, or just trust `docker inspect`
`ctr: cannot access socket`	Docker runs its own containerd	Add `--address /var/run/docker/containerd/containerd.sock`
Storage driver says `overlayfs`, `GraphDriver` is null	You're on Docker 29+	Nothing — that's normal, everything still works

▼ 🆘 **Completely lost? Reset and carry on**

This deletes every container on the machine (images and volumes stay), so you can restart any step from the beginning:

```
docker rm -f $(docker ps -aq) 2>/dev/null; docker ps -a
```

Take-home: ship the Notice Board

Everything from today, on a second app, on your own. Due in 48 hours.

~45 min at home | 🔴 Graded

There's a second service: a **Notice Board API**. A previous intern wrote a Dockerfile for it. It builds, it "works on their machine", and it breaks every single rule you learned today.

Part A: fix the Dockerfile. **Part B:** start the container using only `curl`, like step 4 — no `docker run`, no `docker create`. **Part C:** prove it.

Setup — the app and the intern's broken Dockerfile

```
mkdir -p ~/docker-lab/challenge && cd ~/docker-lab/challenge
echo "flask==3.0.3" > requirements.txt

cat > notice.py <<'EOF'
import argparse, json, os, signal, sys
from datetime import datetime, timezone
from flask import Flask, jsonify, request

app = Flask(__name__)
DATA_FILE = os.path.join(os.environ.get("DATA_DIR", "/data"), "notices.json")

def load_notices():
    try:
        with open(DATA_FILE) as f:
            return json.load(f)
    except FileNotFoundError:
        return []

def save_notices(notices):
    os.makedirs(os.path.dirname(DATA_FILE), exist_ok=True)
    with open(DATA_FILE, "w") as f:
        json.dump(notices, f, indent=2)

@app.get("/notices")
def list_notices():
    return jsonify(load_notices())

@app.post("/notices")
def add_notice():
    body = request.get_json(silent=True) or {}
    text = str(body.get("text", "")).strip()
    if not text:
        return jsonify(error="field 'text' is required"), 400
    notices = load_notices()
    notice = {"id": len(notices) + 1, "text": text,
              "posted_at": datetime.now(timezone.utc).isoformat(timespec="seco
    notices.append(notice)
    save_notices(notices)
    return jsonify(notice), 201

@app.get("/health")
```

```
def health():
    return jsonify(status="ok")

def handle_sigterm(signum, frame):
    print("Notice Board received SIGTERM - exiting cleanly", flush=True)
    sys.exit(0)

signal.signal(signal.SIGTERM, handle_sigterm)

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--port", type=int, default=8000)
    args = parser.parse_args()
    print(f"Notice Board listening on port {args.port}", flush=True)
    app.run(host="0.0.0.0", port=args.port)
EOF

# the intern's Dockerfile. Save it, then fix it.
cat > Dockerfile <<'EOF'
FROM python:3.12
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
USER noticeuser
HEALTHCHECK CMD curl -f http://localhost:80/health
EXPOSE 80
CMD python notice.py --port 8000
EOF
```

There are at least **six** separate mistakes in those nine lines. Today's lab covered every one of them.

The checklist

#	Must be true	Covered in
R1	Multi-stage build, final stage on `python:3.12-slim`	step 9
R2	Dependencies installed **before** the code is copied	step 7.5
R3	Runs as a non-root user	step 9
R4	A `HEALTHCHECK` that works in a slim image, on the right port	step 9
R5	Exec-form `ENTRYPOINT` with an overridable `CMD ["--port","8000"]`	step 7.5–7.6
R6	`docker stop` finishes in under 2 seconds and the log shows the SIGTERM line	step 7.4
R7	Notices survive deleting and re-creating the container (volume `notices-data` at `/data`)	step 8
R8	Limited to 0.5 CPU and 128 MB	step 5
R9	Pushed as `localhost:5000/notice-board:v1`	step 9.3
R10	Container named `notice-board`, created and started **via curl only**, on port 9090	step 4

Your finished deployment must pass all three of these:

```
curl -s -X POST http://localhost:9090/notices -H "Content-Type: application/json" -
curl -s http://localhost:9090/notices; echo
curl -s http://localhost:9090/health; echo
```

▼ 💡 **Hints — open these only after you've tried**

Part A: your step 9 Dockerfile already satisfies R1–R5 for a different app. Put them side by side and compare line by line. Check the base image, the order of the `COPY` lines, whether `noticeuser` is ever actually created, which port the app listens on versus what `EXPOSE` and `HEALTHCHECK` mention, and whether `curl` even exists inside `python:3.12-slim`.

R7 with a non-root user: remember the permission bug in step 9.1. If the image creates `/data` and gives it to your user *before* the volume is ever used, a brand-new volume copies that ownership and there's nothing to fix.

Part B, what should the JSON contain? Don't guess. Create a throwaway container with the normal CLI using the flags you want, run `docker inspect` on it, and read the `Config` and `HostConfig` sections — the field names are all there. Useful ones: `Image`, `ExposedPorts`, `HostConfig.PortBindings`,

`HostConfig.Binds`, `HostConfig.Memory` (bytes), `HostConfig.NanoCpus` (CPUs × 1,000,000,000). Delete the throwaway afterwards.

Part B, the two requests: exactly the pattern from step 4 — `POST /containers/create?name=...` with your JSON, then `POST /containers/notice-board/start`. And remember the one thing the create request will *not* do for you.

Proving R6 and R7: `time docker stop` plus `docker logs` covers R6. For R7, post a notice, delete the container, re-create it through the API with the same volume, then fetch `/notices`.

Bonus marks

- **S1** — build and push a `v2`, then show both tags with `curl -s http://localhost:5000/v2/notice-board/tags/list`.
- **S2** — raise the CPU limit to 1.0 *without restarting*, using `POST /containers/notice-board/update`, and prove it with `docker inspect`.
- **S3** — show the shim process and namespace IDs for `notice-board`, as in steps 3 and 4.

Docker, Hands-On · B.Tech · 2-hour lab

Learn with Gautam Dutta · co-powered by Techify

Stuck on something after the session? Bring the exact command and the exact error message — both, always.